ABCU Advising Program Detailed Pseudocode & Runtime Analysis Course Object Definition

```
Class Course:
  courseNumber : string
  courseTitle : string
  prerequisites : list of strings
```

1. File Loading and Parsing (Detailed)

```
FUNCTION loadCourses(fileName):
  OPEN fileName
  IF file not opened:
    PRINT "Error opening file"
    RETURN

  WHILE NOT end of file:
    READ line
    TRIM whitespace

    SPLIT line by comma INTO tokens

    IF length(tokens) < 2:
      PRINT "Invalid format"
      CONTINUE

    CREATE new Course object course

    SET course.courseNumber = tokens[0]
    SET course.courseTitle = tokens[1]

    INITIALIZE empty prerequisites list

    FOR i FROM 2 TO length(tokens) - 1:
      ADD tokens[i] TO course.prerequisites

    INSERT course INTO data structure
```

```
  CLOSE file
END FUNCTION
```

2. Vector Implementation

```
FUNCTION insertVector(course):
  APPEND course TO vector

FUNCTION searchVector(courseNumber):
  FOR each course IN vector:
    IF course.courseNumber == courseNumber:
      PRINT course.courseNumber
      PRINT course.courseTitle
      IF prerequisites empty:
        PRINT "None"
      ELSE:
        PRINT prerequisites
      RETURN
  PRINT "Course not found"

FUNCTION printSortedVector():
  SORT vector BY courseNumber
  FOR each course IN vector:
    PRINT course.courseNumber, course.courseTitle
```

3. Hash Table Implementation

```
FUNCTION insertHash(course):
  key = hash(course.courseNumber)
  IF table[key] empty:
    table[key] = course
  ELSE:
    ADD course TO bucket list

FUNCTION searchHash(courseNumber):
  key = hash(courseNumber)
  FOR each course IN table[key]:
    IF match:
      PRINT details
      RETURN
  PRINT "Not found"

FUNCTION printAllHash():
```

```
  CREATE list tempList
  FOR each bucket:
    FOR each course:
      ADD to tempList
  SORT tempList
  PRINT all
```

4. Binary Search Tree Implementation

```
FUNCTION insertBST(node, course):
  IF node == null:
    RETURN new node(course)

  IF course.courseNumber < node.courseNumber:
    node.left = insertBST(node.left, course)
  ELSE:
    node.right = insertBST(node.right, course)

  RETURN node

FUNCTION searchBST(node, courseNumber):
  IF node == null:
    PRINT "Not found"
    RETURN

  IF courseNumber == node.courseNumber:
    PRINT details
    RETURN
  ELSE IF courseNumber < node.courseNumber:
    searchBST(node.left)
  ELSE:
    searchBST(node.right)

FUNCTION inorder(node):
  IF node == null:
    RETURN
  inorder(node.left)
  PRINT node.courseNumber, node.courseTitle
  inorder(node.right)
```

5. Menu System

```
WHILE choice != 9:
  PRINT menu
  GET input

  IF 1:
    loadCourses()
  IF 2:
    print sorted
  IF 3:
    search course
  IF 9:
    EXIT
```

6. Runtime Analysis

Reading file: O(n)
Vector insert: O(n)
Hash insert: O(n)
BST insert: O(n log n) avg, O(n^2) worst

7. Recommendation

Binary Search Tree is recommended because it supports sorted output naturally and efficient searching.